

Report

Introduction to ROS2

Dinh-Son Vu

Table of Contents

1. Introduction	4
1.1. Challenges regarding autonomous robots	4
1.2. ROS2, middleware between software and hardware	5
1.3. ROS2 main concept	7
1.4. ROS2: Packages for autonomous robots	8
1.5. Structure of the document	10
2. Development/Desktop Computer setup	13
2.1. Installation of Ubuntu 24.04 on the computer via VirtualBox	13
2.2. Installation of ROS2 Jazzy	15
2.3. Installing additional software on the computer	16
2.4. Installing packages from ROS2	17
3. Getting started with ROS2	18
3.1. Creating a workspace and a package	18
3.2. Creating a launch file	19
3.3. Make your first commit on Github	20
3.4. Clone code from GitHub to your computer	20
4. rmitbot_description package: Description of the robot	21
5. rmitbot_controller package: Adding robot movement	23
6. rmitbot_localization package: knowing robot pose	25
7. rmitbot_firmware package: from simulation to real hardware	27
7.1. Setup PlatformIO to write Arduino code	28
7.2. Arduino sketch for ROS2	30
7.3. Writing the hardware interface of ros2_control	31
7.4. Launching the robot	31
7.4.1. Launching robot simulation	31
7.4.2. Launching robot in real life	32
8. rmitbot_mapping package: slam	33

8.1.	Lidar and slam in simulation	33
8.2.	lidar and slam in real hardware	33
9.	rmitbot_navigation package: planning, obstacle avoidance	34
10.	Tutorial Command Line Interface (CLI) – optional	35
10.1.	Nodes	35
10.2.	Topics	36
10.3.	Services	37
10.4.	Parameters	38
10.5.	Actions	39
11.	Plotjuggler for data visualization – optional	40
12.	Appendix	41
12.1.	Useful terminal script for ros2	41
12.2.	Tutorial SSH	41
12.3.	Tutorial to OOP	41
12.4.	Reference and tutorial for using ROS2	42
13.	ESP32 pinout	43
14.	ESP32 and MicroROS – optional	45
14.1.	Starting a project in PIO and ESP32	45
14.2.	MicroROS architecture	45
14.3.	MicroROS with linorobot	47
15.	Setup rpi5 for ros2	48
16.	Setup a joystick with ros2	50
17.	setup docker – tba	51

1. Introduction

This is an introduction to ROS2, which is a middleware running on a development computer or an on-board computer (Raspberry Pi, Jetson). ROS2 comes with a steep learning curve, but it is useful to build robots with intelligence, contrary to robots that are teleoperated only.

- **ROS2** stands for Robotic Operating System, and it is the second version. Operating system is quite confusing because it does not include a graphical interface just like Windows or MacOS, but it mainly uses terminal command.
- **A middleware** is a software that stands in between the hardware and the software. ROS2 is similar to a driver for peripherals: when you plug a mouse to a computer, it just works instantaneously. The drivers are installed automatically as soon as the device is plugged in, and you do not write any code to convert the optical flow sensor (mouse sensor) to movement in the x-y direction. Moreover, your computer understands the information provided by the mouse directly thanks to the standardization of the communication protocol via USB.

1.1. Challenges regarding autonomous robots

In French, with English subtitles available. The laboratory is Norlab, which is focusing on autonomous robots in northern harsh environment.

- History of autonomous vehicles: <https://youtu.be/-vKPvJXooNs?si=iUMQm3fQmWnBT1B5>
- What is a mobile robot? https://youtu.be/PJn7bch13Yo?si=kv2qXKYrSWB_zeJb

A robot needs sensors to perceive the environment, algorithms to treat these information, and actuators to act accordingly.

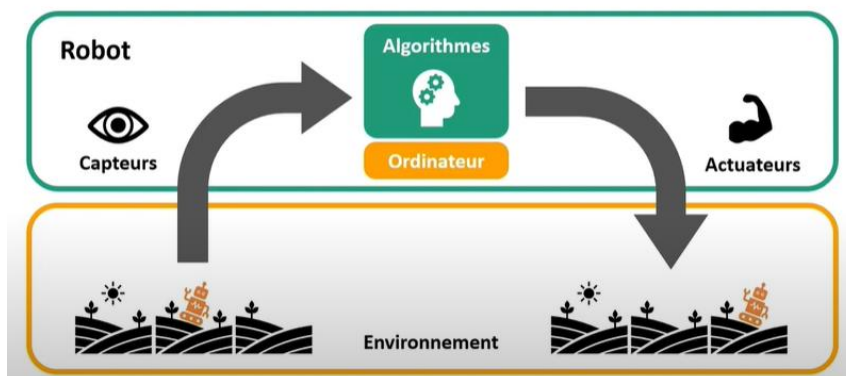


Figure 1: An autonomous robot requires sensors, actuators, and algorithms

A robot has several peripherals (generic names for sensors and actuators, similar to peripherals for a computer, e.g., mouse, keyboard, screen, speakers, microphone, etc.), with different algorithms that are intense and quite consuming to develop, such as:

- Code interfacing between the hardware to the computer (i.e., a driver)
 - o Sensors: Lidar, radar, sonar, camera, GPS, IMU, encoder

- Actuators: DC, BLDC, stepper
- Algorithms to interpret the sensors' data:
 - Computer vision with camera sensor
 - Mapping/SLAM with Lidar, radar, sonar sensor
 - Positioning/navigation with GPS, IMU, and encoder
 - Digital twin – Visualization/simulation of the robot on the computer

For simple tasks, such as teleoperation of a wheeled robot, Arduino/esp32 are fine. Command from the joystick are translated to command to the robots' motors. However, complex tasks involve more hardware, communication protocol, and computation. For instance, a typical autonomous robot involve four wheels, four encoders, one depth camera with computer vision algorithm, lidar with mapping (slam), swarm/decentralized communication for multiple robots, texture mapping with depth 3D camera ...: there is no way to implement this into an Arduino robot.

1.2. ROS2, middleware between software and hardware

In French, with English subtitles available.

- Why ROS2? <https://youtu.be/vAb5SnaJbF0?si=bhGIA70abbXpxJWM>
- Camera and vision: <https://youtu.be/hfCB2IRdQmg?si=FS-GwnXfUbptW-K2>
- Modeling and control: <https://youtu.be/UhVIXgckmv4?si=St-mZjxOBteLqQ2Q>
- Lidar and SLAM: <https://youtu.be/ngcmNkzTbM4?si=djCLtcyxINUTXqgH>

The development of robotic systems is a worldwide activity. Thus, robotic engineers share common problems, such as drivers for hardware, navigation, SLAM, computer vision, etc.

- ROS2 is based on the systems of **nodes** (pieces of individual code) that are communicating together with channels called **topics**.
- It uses two programming languages: Python for prototyping, analysis, and communication between nodes. The other language C++ is used for real-time (microcontroller).
- The greatest benefit of ROS2 is code sharing: you can decide to write down the computer vision algorithm all over again, or to use a library that has already been developed: it requires works regarding the inputs, outputs, and parameters of the packages.

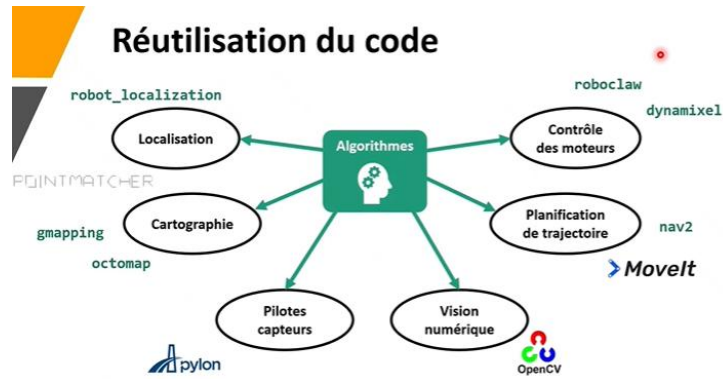


Figure 2: Autonomous vehicles must handle several tasks: mapping, navigation, computer vision, etc. Code sharing become even more important as the robot is becoming more autonomous and intelligent.



Figure 3: ROS2 aims at creating a standard for all robotic platforms: they thus can be summarized as black box receiving inputs and transmitting outputs.

Outils intégrés

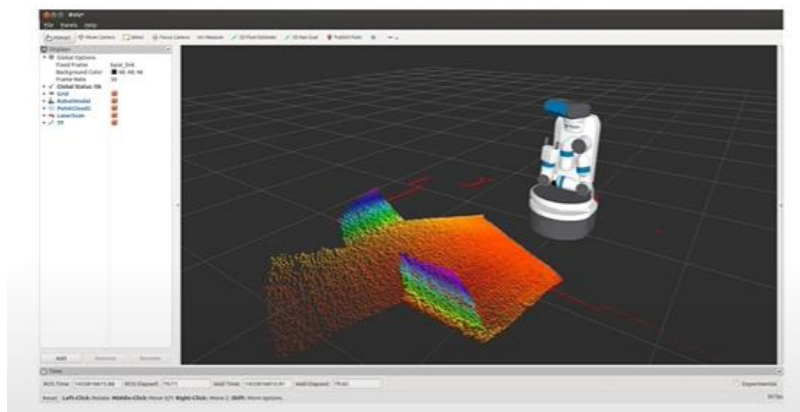


Figure 4: Digital Twin (Rviz and Gazebo) - ROS2 offers tools for visualization of data coming from the hardware, which is time consuming if one needs to develop it entirely.

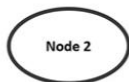
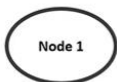
1.3. ROS2 main concept

ROS2 aims at creating a standard for different hardware, software, and engineers. Thus the ROS2 framework should be understood first.

- **Node**: it is a bloc of code. For example, inverse kinematics takes Cartesian position x,y,z as an input and output joint angles.
- **Topic**: communication messages between nodes. Nodes subscribe to a topic (it receive information from the topic as an input) or publish to a topic (it sends their output information to the topic).
- ROS2 uses the **Terminal** extensively: one must become comfortable with command line.
- **URDF** stands for Unified Robot Description Format: it describes the robot dimension and types of joint in an xml format. This is an essential part of ROS2, as one must perform simulation and visualization of the actual robot.
- **Packages** is a collection of nodes, topics, and libraries that performs a specific task, e.g., computer vision package, SLAM package, etc. Thus, one must understand the concept of cloning, compiling, and utilizing a package.
- **Launch files** is a Python script to launch of the required packages for your robot.

Nodes

- Élément de base de ROS
- Bloc de code qui effectue une tâche spécifique
- Indépendant des autres nodes



Topics

- Canaux de communication entre les nodes
- Échange un type de message précis
- Les nodes "s'abonnent" et "publient" sur des topics



Figure 5: Nodes and Topics are the cornerstone of ROS2.

URDF

- Fichier de description du robot
- Définition des TFs entre les modules
- Visualisation 3D
- Caractérisation physique des modules pour la simulation (ex: inertie, masse, volume)

Figure 6: URDF is essential for describing a robot in ROS2 and especially for RVIZ

Packages

- Regroupement de code ROS réutilisable
- Peut contenir :
 - Nodes
 - Bibliothèques externes
 - Fichiers de configuration
 - URDF
 - Messages personnalisés
 - +++
- Clone -> Compile -> Utilise



Figure 7: ROS2 is about integrating packages together

Launch files

- Configuration du système
- Lancer plusieurs nodes en même temps

```
ros2 launch <package_name>
<launch_file_name>
```

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='turtlesim',
            namespace='turtlesim1',
            executable='turtlesim_node',
            name='sim1'
        ),
        Node(
            package='turtlesim',
            namespace='turtlesim2',
            executable='turtlesim_node',
            name='sim2'
        ),
        Node(
            package='turtlesim',
            executable='mini1',
            name='mini1',
            remappings=[
                ('/input/pose', '/turtlesim1/turtle1/pose'),
                ('/output/cmd_vel', '/turtlesim2/turtle1/cmd_vel')
            ]
        )
    ])

```

Figure 8: Launch file: a bit similar to MATLAB script: you launch the nodes

1.4. ROS2: Packages for autonomous robots

The following GitHub has an excellent architecture regarding the code architecture. The generic name used for the robot in this document is rmitbot. Feel free to modify its name. Some of the tools are not presented yet, but will come in the upcoming sections.

<https://github.com/ AntoBrandi/Bumper-Bot>

<https://github.com/ros-mobile-robots/diffbot> (this is ROS1 though)

<https://automaticaddison.com/naming-and-organizing-packages-in-large-ros-2-projects/>

<https://github.com/aws-deepracer/aws-deepracer/blob/main/introduction-to-the-ros-navigation-stack-using-aws-deepracer-evo.md>

Name	Description
1. rmitbot_description	- contains urdf and gazebo configuration - uses gazebo and rviz (package for visualization) - The virtual robot cannot move yet.
2. rmitbot_controller	- uses ros2_control and ros2_controller (package for controlling robot) - interface the joystick/keyboard controller to the joints of the robot - the virtual robot can move, its motion can be measured with odometry. Adding an IMU would attenuate the error due to slippage.
3. rmitbot_localization	- uses robot_localization (package for ekf) - performs sensor fusion (ekf) between odometry and imu - the virtual robot can move, and its pose can be displayed
4. rmitbot_firmware	- hardware interface from the ros2_control package - transition between the virtual environment to the real hardware.
5. rmitbot_mapping	- Create a map of the environment with slam_toolbox
6. rmitbot_navigation	- path planning (a_star): path to get to the target - motion planning: path following with PID - navigation: decision tree relative to known and unknown obstacles

The workspace should look like the following:

rmitbot_ws

build, install, log, .vscode: → Folders are automatically created while setting up the ws

src → contains all the packages listed below

```

rmitbot_bringup
rmitbot_controller
rmitbot_description
rmitbot_localization
rmitbot_firmware
rmitbot_mapping
rmitbot_navigation

```

1.5. Structure of the document

The document will follow a structure similar to the package that needs to be implemented and is given as follows:

1. Introduction to ros2
2. Configuration of the dev computer: install VirtualBox, Ubuntu 24.04, ros2 Jazzy, and additional software like vscode, github.
3. Getting started with ros2: nodes, topics and CLI
4. Package rmitbot_description
5. Package rmitbot_controller
6. Package rmitbot_localization
7. Package rmitbot_firmware
8. Package rmitbot_mapping
9. Package rmitbot_navigation

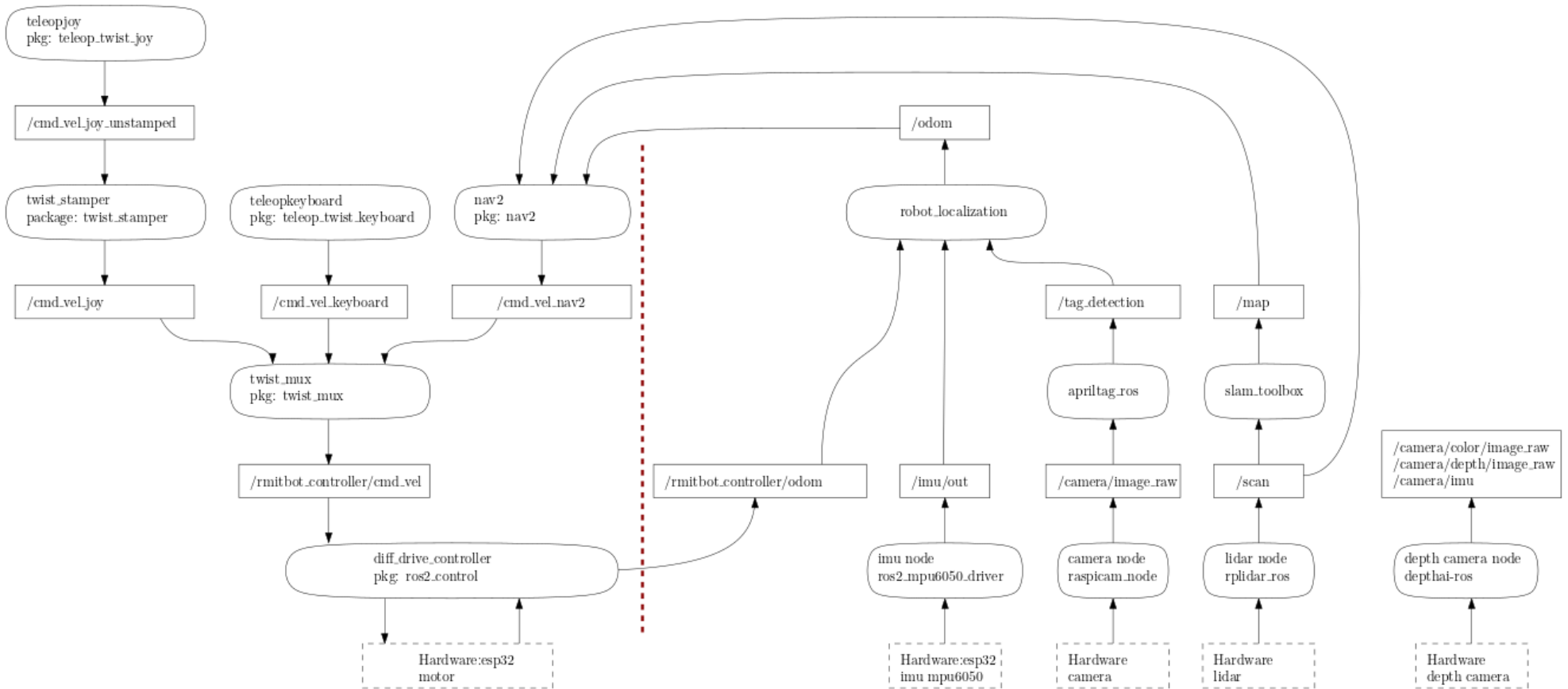


Figure 9: ros2 framework for a standard autonomous robot

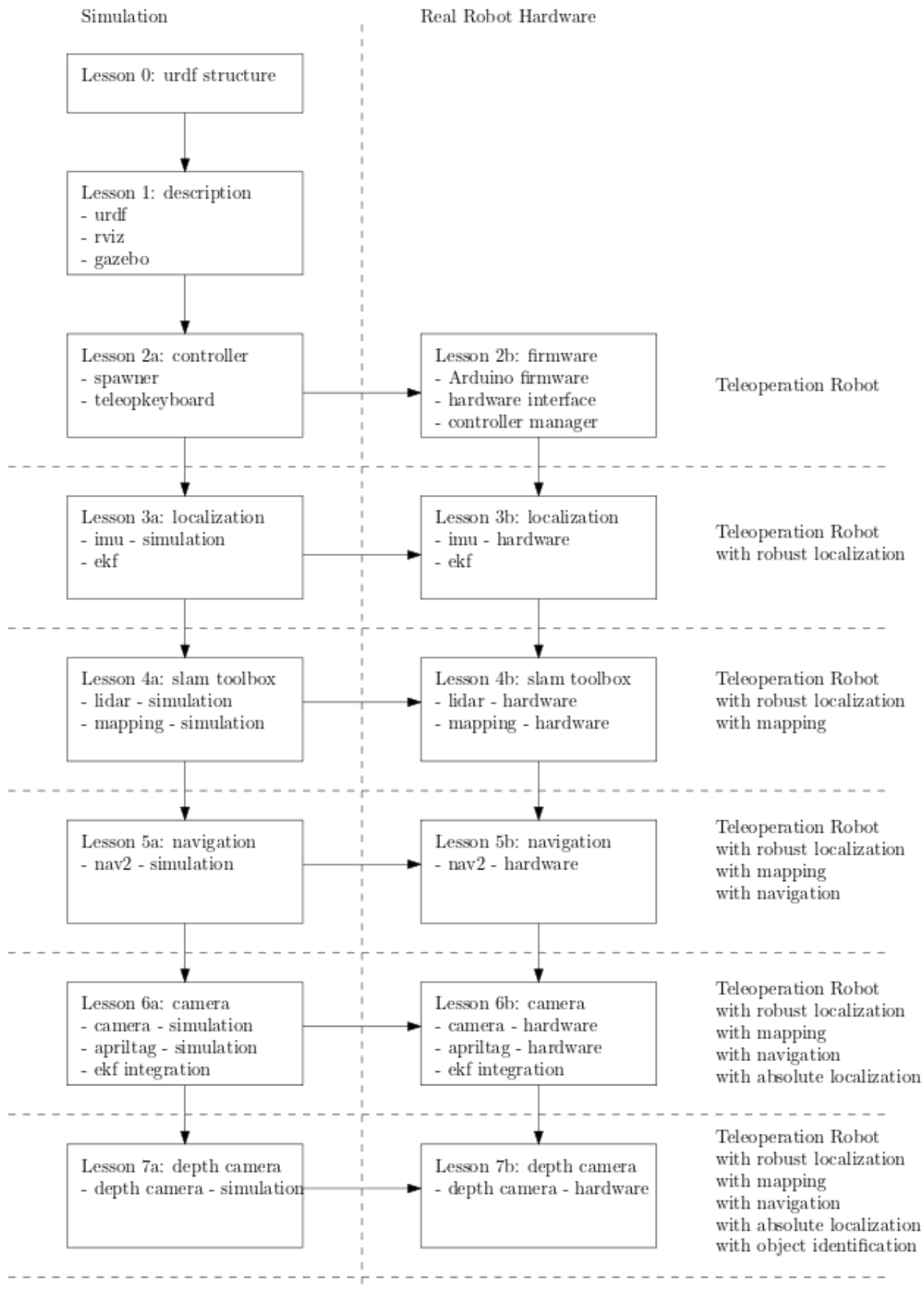


Figure 10: Structure of the lesson, including simulation and hardware manipulation.

2. Development/Desktop Computer setup

The development computer corresponds to the desktop/laptop computer. ROS2 works better in **Ubuntu**. To avoid dual boot, you may use a virtual machine, such as VirtualBox.

2.1. Installation of Ubuntu 24.04 on the computer via VirtualBox

Please follow the instructions:

- Install the latest version of VirtualBox: <https://www.virtualbox.org/>
- Download Ubuntu 24.04.2 LTS: <https://ubuntu.com/download/desktop>
- Follow the tutorial for configuring the virtual machine:

<https://www.theroboticspace.com/blog/How-To-Install-ROS-2-in-Ubuntu-22-04-VM-On-Windows/>

Some parameters from VirtualBox should be changed:

- If there is a black screen at startup using Oracle VM:

Display → Graphics Controller → VBoxSVGA.

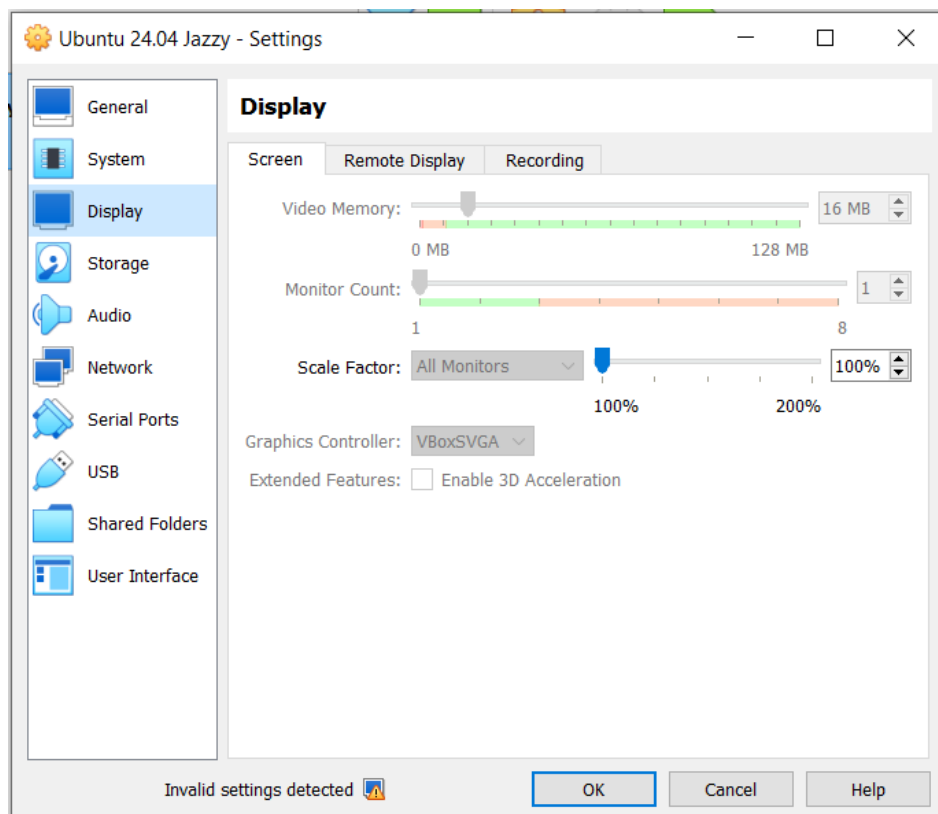


Figure 11: Display → Screen → Graphics Controller → VBoxSVGA

- For the Wi-Fi to work, add an Adapter 1 and Adapter 2 as shown below:

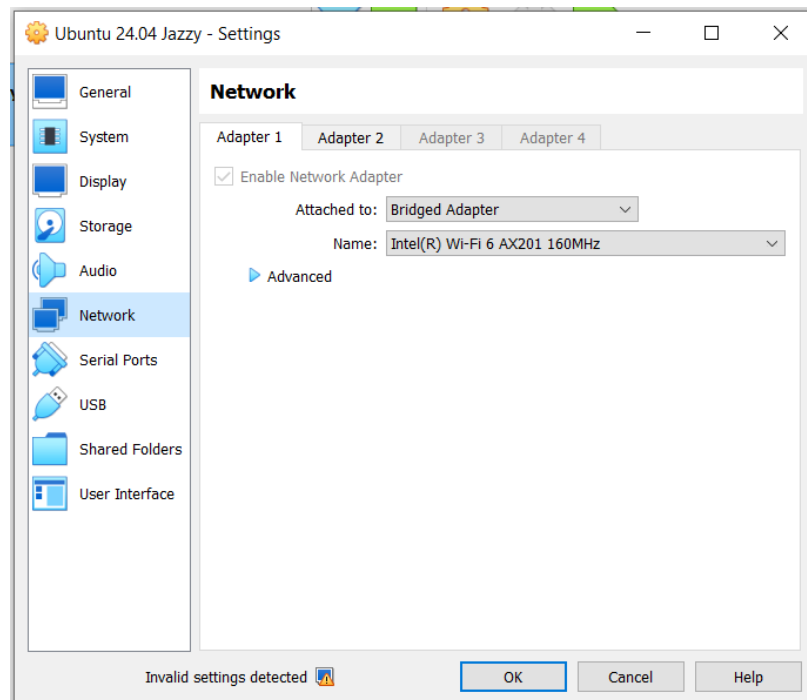


Figure 12: Network → Adapter 1 → Bridged Adapter

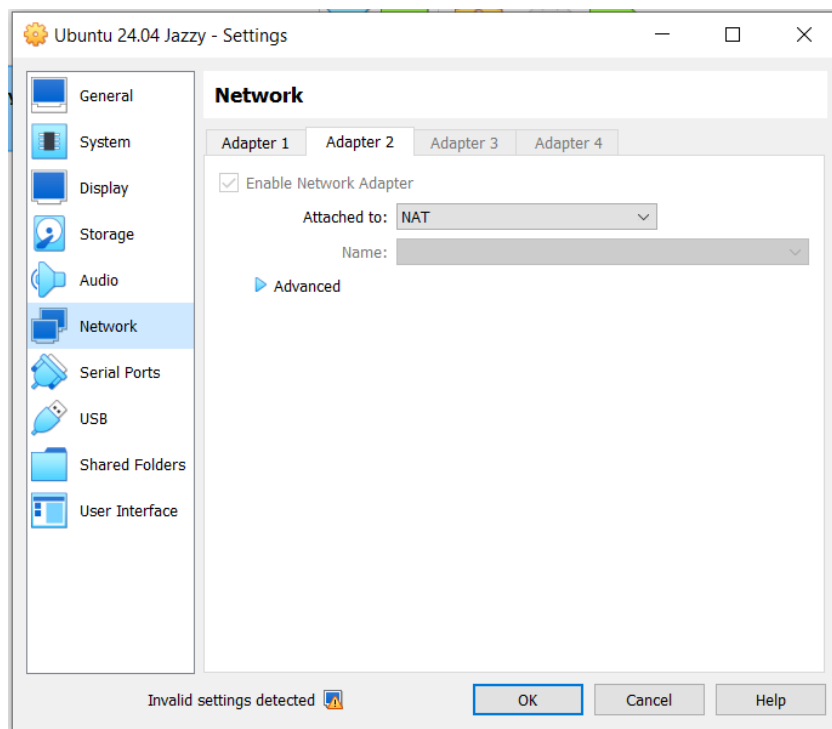


Figure 13: Network → Adapter 2 → Attached to NAT (for wired connection)

- If your computer has USB3, use it, otherwise ubuntu freeze while disconnecting the esp32. For flashing purposes, USB must be configured as follows.

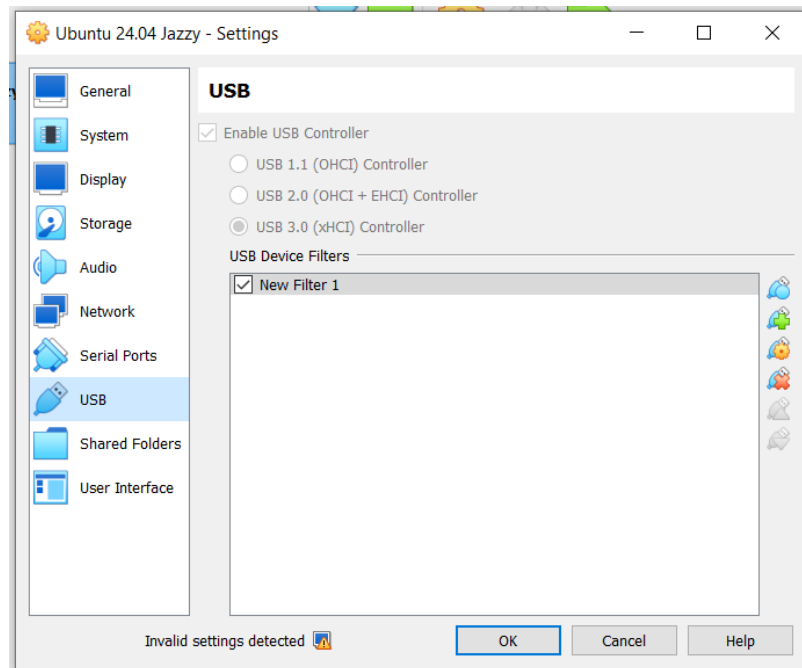


Figure 14: USB → USB 3.0 → Add New Filter

2.2. Installation of ROS2 Jazzy

ROS2 should be installed in Ubuntu 24.04, that is running on VirtualBox. Follow the tutorial:

<https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html>

All commands must be run in a terminal. During each step of the installation process, make sure that there is no error message. One error message that can arise is No_Public Key: please look at: <https://robotics.stackexchange.com/questions/114709/i-keep-getting-this-error-whenever-i-try-to-install-universe-or-any-ros2-file>

maybe you added the repository before adding the ROS GPG key:

```
sudo curl -sSL raw.githubusercontent.com/ros/rosdistro/master/ros.key -o /usr/share/keyrings/ros-archive-keyring.gpg
```

2.3. Installing additional software on the computer

The following software helps with the development of the robot:

- GIT: push code on GitHub. You will need to create an account.
- Terminator: an app to organize terminal on ubuntu (much better than the Ubuntu Terminal)
- VSCode: best IDE for programming in python, cpp, html, etc.

In VSCode, several extensions should be installed:

- C/C++ and extensions pack
- CMAKE from twxs. For auto-completion
- Python
- XML and XML tools
- ROS: from Microsoft. Extension for developing ROS node and URDF
- PlatformIO: from PlatformIO. Best extension for programming properly microcontroller

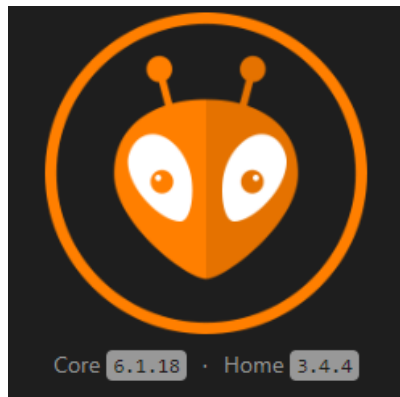


Figure 15: Please verify that the PlatformIO (PIO) core is 6.1.18 or above

Tutorial: Getting Ready to Build Robots with ROS:

<https://youtu.be/2lIV3dRvHmQ?si=jJiYhZ3DYwAhKd-M>

Command to install software from the terminal in Ubuntu:

Git: `sudo apt install git`

OpenSSH: `sudo apt install openssh-server`

VSCode: `sudo snap install --classic code`

PlatformIO is an extension of VSCode. Please find the following tutorial to get started coding with PlatformIO and microcontroller:

<https://randomnerdtutorials.com/vs-code-platformio-ide-esp32-esp8266-arduino/>

2.4. Installing packages from ROS2

Reminder: ROS2 is a middleware: it links hardware and software together. For instance, you plug in a joystick controller on a Windows PC, and it will work automatically, without writing any code to convert potentiometer data from the joystick to the serial communication and to the desired application on the computer. A package in ROS2 is a software, for instance, for visualization of the robot, navigation, computer vision, etc. Useful packages to install with ROS2 are:

- ros2_control: to control hardware (motor) or simulator:
`sudo-get install ros-jazzy-ros2-control`
- ros2_controller: to perform the inverse kinematics (from joystick input to wheel speed output)
`sudo-get install ros-jazzy-ros2-controller`
- xacro: xml macro for describing the robot architecture
`sudo-get install ros-jazzy-xacro`
- Gazebo: simulator (quite heavy installation): physic engine
`sudo-get install ros-jazzy-ros-gz*`
- Additional plugins for the control of the robot in Gazebo simulator
`sudo-get install ros-jazzy-*-ros2-control`
- Joint state publisher
`sudo-get install ros-jazzy-joint-state-publisher-gui`
- For the tutorial simulation: turtlesim
`sudo-get install ros-jazzy-turtlesim`
- Robot localization
`sudo-get install ros-jazzy-robot-localization`
- Joystick
`sudo-get install ros-jazzy-joy`
`sudo-get install ros-jazzy-joy-teleop`
- TF Transformation
`sudo-get install ros-jazzy-tf-transformations`
- Python pip
`sudo apt-get install python3-pip`
- python 3D package
`pip install transforms3d` (sometimes it may not work)
`sudo apt install python3-transforms3d`
- serial lib
`sudo apt install libserial-dev`

3. Getting started with ROS2

There are several concepts to be handle:

- Create a workspace and a package
- Push code on Github
- Clone code from Github

Refer to lesson1_ws on Github: https://github.com/ROS2-AutoBot/lesson1_ws.git

3.1. Creating a workspace and a package

Install the compiler colcon:

```
sudo apt install python3-colcon-common-extensions
```

A workspace is just a folder that will contains packages. Make a directory called lesson1_ws, with a folder called src:

```
mkdir -p lesson1_ws/src
```

Packages are stored in the src folder. The nodes of a package can be code in python or cpp. For the moment, we will focus on python code. Change directory to the src folder and build a new package called my_package:

```
cd lesson1_ws/src
```

```
ros2 pkg create --build-type ament_python my_package → for python nodes
```

```
ros2 pkg create --build-type ament_cmake my_package → for cpp nodes (for reference)
```

A folder called “my_package” has been created, with the following components: include, src, CMakeLists.txt, package.xml. Verify that your package is available

```
ros2 pkg list
```

For the moment, the newly package created does nothing (there is no node/code inside).

Change directory back to the workspace folder and build the workspace It will create three folders: build, install, and log. Then source the workspace, to let the computer know where you are starting to work on your computer.

```
cd ~/lesson1_ws
```

```
colcon build
```

```
source install/setup.bash → source the workspace
```

```
. install/setup.bash → this command line can be used as well.
```

Optional: At some point, you may be tired of sourcing your workspace every time you are opening a terminal. In this case, you can add definitively the workspace to the main bashrc.

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc      → Add the global ros2
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc  → Add this particular directory
source ~/.bashrc                                          → Apply change
```

Bw careful, do not do this if you have several package of the same name in different workspace.

3.2. Creating a launch file

Tutorial: https://youtu.be/xJ3WAs8GndA?si=W_VEqCKo5m_yVBqb

Allows to launch several nodes with their associated parameters.

Go to the src of your workspace and create a package. You may change the name of the robot:

```
ros2 pkg create my_awesome_robot
```

You may remove include and src folder, since you are not writing a node

```
rm -rf include/
rm -rf src/
```

Create a launch folder:

```
mkdir launch
```

Create a launch file in the launch folder

```
cd launch/
touch demo.launch.py
```

Come back to the src folder and start VSCode

```
cd ../../
code .
```

Follow the step in the video to modify the launch file. Build the package from the workspace folder, source the workspace, and launch the launch file:

```
colcon build
source install/setup.bash
ros2 launch my_robot demo.launch.py
```

ros2 launch <package_name> <launch_file> ros2 launch gazebo_tutorial gazebo.launch.py	Launch a launch file (python)
ros2 pkg list ros2 pkg executables ros2 pkg executables <pkg_name>	Packages available in ros2 Executables (e.g., nodes) available in ros2 Executables of a particular package

Warning: you may need to install xterm to use teleop keyboard in a new terminal.

3.3. Make your first commit on Github

Make sure you have installed Git (please refer to the setup of your dev computer). Create a new repository on your GitHub account – for example lesson1_ws. Do not add a readme file yet.

On VSCode, go to your folder containing the code and additional folders:

- Initialization: `git init`
- Add everything: `git add .`
- Make a commit: `git commit -m "first commit"`
- Create a branch: `git branch -M main`
- Remote path (change the address to the repository you have created):
 - `git remote add origin https://github.com/ROS2-AutoBot/lesson1\_ws.git`
- Sometimes, there is already an origin, check with
 - `git remote -v`
- You may change it with the following cli
 - `git remote set-url origin https://github.com/ROS2-AutoBot/lesson1\_ws.git`
- Push it to GitHub: `git push -u origin main`

Normally, the entire folder should be copied on your online repository.

3.4. Clone code from GitHub to your computer

This is an essential task: since most of the code have been developed already, the aim is to clone the correct code and adapt it to your needs. In a terminal, navigate to the folder/workspace where you would like to clone the repository.

```
cd <workspace>
```

Clone the repository:

```
git clone https://github.com/ROS2-AutoBot/lesson1\_ws.git
```

Change back to the workspace directory, compile the project, and source the workspace:

```
cd <workspace>
```

```
colcon build
```

```
. install/setup.bash
```

If there is a compiling error, you may have to remove build, install, log, and colcon build.

4. rmitbot_description package: Description of the robot

This section cover the following aspect:

- urdf: how to define a robot in ros2
- rviz: how to visualize the robot in ros2
- gazebo: how to simulate the robot and its environment in ros2

rviz and gazebo looks like similar tools, but they have different purposes:

- rviz is just for visualization: it subscribes to topics, such as robot frames (TF), lidar scan.
- Gazebo is for simulation: the robot will appear in gazebo, but also the entire external environment (room and obstacles), and its physics, such as friction of the wheels. When the hardware is not available, gazebo is handy to continue coding. Usually, gazebo is used at the early stage to make sure that all the nodes for communication are ready, then the code is deployed to the real robot. The encoder, the imu, the lidar, and even the camera feed and depth camera can be simulated in gazebo, which makes it very practical to test apriltag_ros.

Install gazebo for jazzy:

```
sudo apt-get install ros-jazzy-ros-gz
```

You may clone the repository to get started with this lesson:

```
git clone https://github.com/ROS2-AutoBot/lesson2\_ws.git
```

Launch rviz and gazebo

```
ros2 launch rmit_description display.launch.py
```

```
ros2 launch rmit_description gazebo.launch.py
```

At the end of this lesson, the robot can be visuazlised in rviz (i.e., the tf) and gazebo, but the robot cannot move. This is covered by rmibot_controller package in the next section.

Instead of cloning the repository directly, you may want to do the process manually to get familiar with command line interface (CLI).

Create a workspace:

```
mkdir -p lesson2_ws/src
```

Change directory to src

```
cd src
```

Create a package called rmitbot_description in the src directory

```
ros2 pkg create --build-type ament_cmake rmitbot_description
```

Explanation:

- The folder include is created by default. It is empty since no node is developed.
- The folder launch will contain the launch file, to launch several nodes at the same time:
 - o display.launch.py: visualization in rviz
 - o gazebo.launch.py: simulation in gazebo
- The folder meshes contains stl file for the rendering of the robot.
- The folder rviz contains a configuration file for rviz
- The folder src is created by default. It is empty since no node is developed
- The folder urdf will contain the description of the robot in a xml format
- The CMakeLists.txt must be modified to add the path of the folder
 - o launch, urdf, rviz, meshes must be added to the path
- The package.xml must be modified to add the executable dependencies.
 - o robot_state_publisher, joint_state_publisher_gui, rviz, ros2launch for rviz
 - o ros_gz_sim for gazebo

There are packages that will be used (natively installed with ros2):

- robot_state_publisher: Requires an urdf file as an input. It performs the transform (tf) between robot's links. These tf datatype are broadcasted and used by rviz2.
- joint_state_publisher_gui: provides a real-time joint state data, which is used by the robot_state_publisher to calculate the tf and broadcast them.
- rviz : rviz can be used to analyze topic, especially graphical one (lidar, TF2, etc.)
takes the tf broadcasted by the robot_state_publisher and the joint state from the joint_state_publisher_gui to render the robot

Gazebo is a useful tool for simulation of physics and digital twin (similar to unity, but gz is more suited fo ros2). It can replace the real robot and publish topics on Rivz. It requires the urdf file to have visual, collision, and inertia (the latter can be quite tough to obtain manually, Solidworks and Fusion can handle it. SolidWorks→URDF export or Fusion→URDF). Gazebo also requires a file for dynamic coefficient: friction, stiffness, damping.

Optional: at some point, you may want to convert from Fusion360 to urdf:

<https://youtu.be/ZFo6wPXjeQ?si=YaFbOM3yTqNWv3-B>

5. rmitbot_controller package: Adding robot movement

Install ros2_control

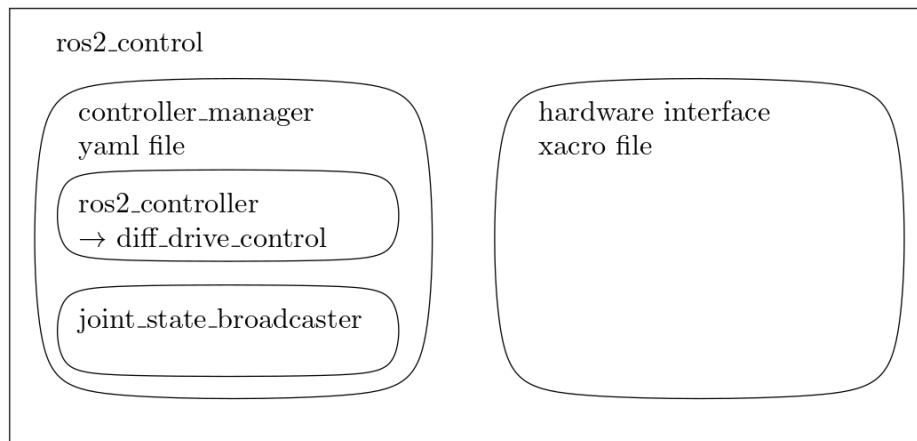
```
sudo apt install ros-jazzy-ros2-control
```

Install ros2_controller

```
sudo apt install ros-jazzy-ros2-control ros-jazzy-ros2-controllers
```

You can git clone the workspace to follow the lesson:

```
git clone https://github.com/ROS2-AutoBot/lesson3\_ws.git
```



The workspace includes the following packages:

- `rmitbot_description`: this package has been developed previously. However, it must be updated with the following file:
 - o `urdf/rmitbot_ros2_control.xacro`: this xacro file defines the hardware that will be controlled (`command_interface`) and the information from the hardware (`state_interface`). Also, It tells if gazebo simulation or the real hardware should be used. If gazebo is used, it loads the required plugin. For the moment, we will focus on the gazebo simulation only. The real robot will be dealt in the later section.
- `rmitbot_controller`
 - o `config/rmitbot_controller.yaml`: configuration of the `ros2_controller` package. In particular, it defines the `controller_manager`, which loads two nodes at startup:
 - `joint_state_broadcaster`: this is not really doing any control, it is a node that is broadcasting `/joint_state`, which is essential for `robot_state_publisher`, which broadcasts the `tf` of the robot (essential for `rviz` visualization)
 - `rmitbot_controller`: it loads the node `diff_drive_base_controller`. This node publishes the topic `/rmitbot_controller/odom`, which is the main way of getting the pose of the robot from encoders. It subscribes to a twist `/rmitbot_controller/cmd_vel` and perform the inverse kinematics, to send the

command to the motors of the robot (whether to gazebo or to the hardware interface).

- teleopkeyboard: Use of keyboard to control the robot in gazebo.

You need to install xterm as an external terminal for the teleoperation keyboard.

```
sudo apt install ros-jazzy-teleop-twist-keyboard
```

```
sudo apt install xterm
```

- teleopjoystick: Use of joystick to control the robot in gazebo.

```
sudo apt install ros-jazzy-joy
```

```
sudo apt install ros-jazzy-teleop-twist-joy
```

```
sudo apt install ros-jazzy-twist-stamper
```

- twistmux: use the keyboard, joystick, and later the navigation from nav2 to control the robot.

```
sudo apt install ros-jazzy-twist-mux
```

Launch rviz and gazebo

```
ros2 launch rmit_description display.launch.py
```

```
ros2 launch rmit_description gazebo.launch.py
```

Launch controller setup and keyboard teleop

```
ros2 launch rmit_controller controller.launch.py
```

```
ros2 launch rmit_controller teleopkeyboard.launch.py
```

Now, We can use `ros2_control` to move our robot in the simulated environment. Later, we will see how to go from simulation to real hardware. One should pay attention that the parameters `use_sim_time` has been set to true for all nodes. When real hardware will be used, then `use_sime_time` will not be used and be set to false.

Note: the package publishes a topic called `/rmitbot_controller/odom`, and it also broadcasts a tf called `/odom`. Publishing a topic and broadcasting a tf is similar, but differs in the following aspects:

- Publishing a topic in the general method for communication in ros2.
- Broadcasting a tf is specialized for pose of the robot

6. rmitbot_localization package: knowing robot pose

Install the package:

```
sudo apt install ros-jazzy-robot-localization
```

Understanding tf tree will be essential, install the tf2 tool package:

```
sudo apt install ros-jazzy-tf2-tools
```

You can git clone the lesson 4:

```
git clone https://github.com/ROS2-AutoBot/lesson4\_ws.git
```

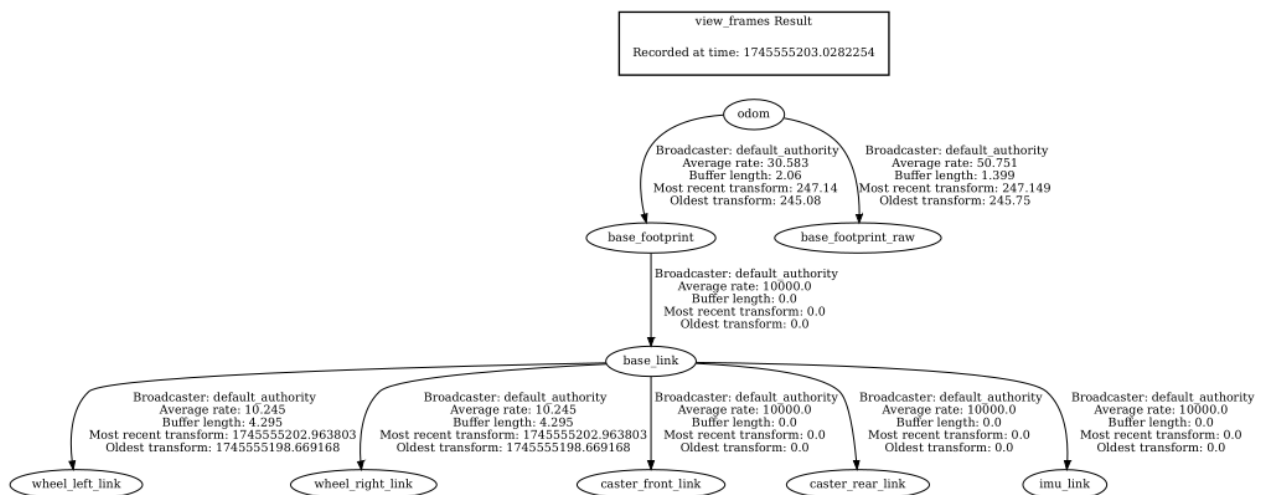
Important note:

- When using the robot_localization/ekf package, in controller.yaml, make sure that the parameter `enable_odom_tf` is set to `FALSE`, otherwise there will be a conflict on the tf publication and the tf odom → base_footprint will “jerk” in rviz. The diff_drive_controller still publishes the TOPIC `/rmitbot_controller/odom`, but the ekf node should be the only one to broadcast the tf odom→base_footprint. The ekf will publish the topic `/odometry/filtered`, which is the fused odometry from different sources.

Later, you will need to visualize the tf tree by running the following cli:

```
ros2 run tf2_tools view_frames
```

```
evince <frame.pdf>
```



The localization of a robot is one of the hardest challenge in mobile robot. The encoder of the robot give the pose (position and orientation) of a robot, but it can drift due to slippage of the wheel, inaccuracy of the encoders, error of the wheel and base dimensions, etc. Usually, the pose estimate is given with:

- encoders: also called odometry or dead reckoning
- imu: inertial measurement unit
- gps (outdoor): absolute positioning of the robot
- lidar odometry: LiDAR odometry estimates motion by aligning successive LiDAR scans
- visual odometry: works better with depth camera and is similar to lidar odometry.
- Visual apriltag: tag are used to give absolute positioning of the robot

Tasks done in each package:

- rmitbot_description:
 - o Add a new IMU in the robot_description urdf.xacro (loading stl file graphics)
 - o Add the stl file in the meshes folder (already inside)
 - o Add gazebo.xacro information (add plugin imu, add noise for more realistic physics)
 - o Add gz_ros2_bridge for the imu (clock@rosgraph already added to use sim_time)
 - o Add dependencies in the package.xml file
- rmitbot_controller:
 - o Nothing added.
- rmitbot_localization: new package
 - o ekf.yaml: defines the parameters for the extended Kalman filter (core of the robot_localization)
 - o local_localization.launch.py: launch file to start the local localization (should be renamed localization only later on)

Launch rviz and gazebo

```
ros2 launch rmit_description display.launch.py
```

```
ros2 launch rmit_description gazebo.launch.py
```

Launch controller setup and keyboard teleop

```
ros2 launch rmit_controller controller.launch.py
```

```
ros2 launch rmit_controller teleopkeyboard.launch.py
```

Launch local localization

```
ros2 launch rmit_localization local_localization.launch.py
```

7. rmitbot_firmware package: from simulation to real hardware

This topic is quite dense as we are moving from the simulation to the real hardware. Two launch files will be created:

- simrobot.launch.py: it will launch the following launch files:
 - o display.launch.py
 - robot_state_publisher. Requires use_sim_time:=true
 - rviz
 - o gazebo.launch.py
 - gazebo and its dependencies
 - o ros2_control. Requires use_sim_time:=true
 - o

So far, the robot was controlled in simulation only (gazebo). Now it is desired to control and measure the real hardware. The firmware can be divided into two sections:

- Arduino firware: the code that will be flashed on the microcontroller. ESP32 is the preferred microcontroller, along with the Arduino framework. The Arduino sketch/firmware on the microcontroller communicate with the rpi5 containing ros2 through the serial interface as follows:
 - o Arduino prints estimated velocities, IMU on the serial com for the ros2 node to read
 - o Arduino reads velocity command from the ros2 node that is writing it.
- Hardware interface firmware: it corresponds to the node that is reading and writing on the serial communication:
 - o The node reads the estimated velocities and the IMU
 - o The node writes the velocity command

Instead of writing a custom node that read/write on the serial interface, it is preferable to use ros2_control and to write a hardware interface, which is a bit more complicated:

- Communication is much faster (almost real-time : 1kHz).
- It has life cycle management (init, config, kill nodes).

Please refer to lesson5_ws: https://github.com/ROS2-AutoBot/lesson5_ws.git

Important note:

- When using the real robot, gazebo should NOT be used: only rviz should be used for visualization. You don't need simulation anymore since you have the real hardware
- When using the real robot, make sure that the parameter use_sim_time is set to false: while launching a file, you can use a command line for instance:
`ros2 launch rmitbot_description display.launch.py use_sim_time:=false`

7.1. Setup PlatformIO to write Arduino code

It is preferable to use PlatformIO (PIO) than the Arduino IDE to write the Arduino sketch. A tutorial to get started with PIO can be found as follows:

<https://randomnerdtutorials.com/vs-code-platformio-ide-esp32-esp8266-arduino/>

Create a directory/workspace. The terminal with `mkdir` or the file explorer can be used.

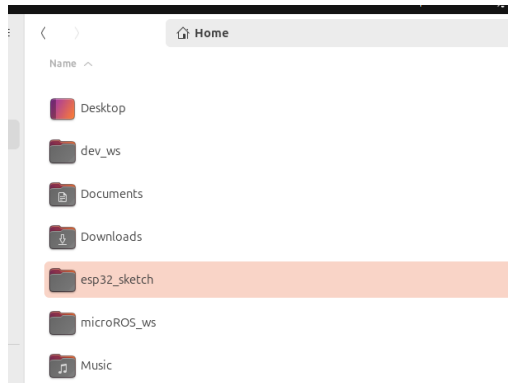


Figure 16: Create a folder called `esp32_sketch`

If you are using the file explorer, with a right-click the option `Open in Terminal` is available.

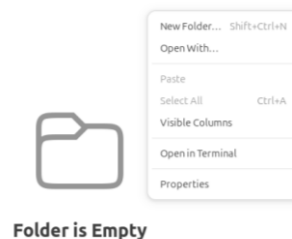


Figure 17: Go inside the folder and select `"Open in Terminal"`

Where the terminal is open, typing `"code ."` open VSCode and open the directory inside VSCode.

```
vudinhso@vudinhso-VirtualBox: ~/esp32_sketch
vudinhso@vudinhso-VirtualBox:~/esp32_sketch$ git clone https://github.com/01-LEDBlinking/...
Cloning into '01-LEDBlinking'...
remote: Enumerating objects: 28, done.
remote: Counting objects: 100% (28/28), done.
remote: Compressing objects: 100% (21/21), done.
remote: Total 28 (delta 6), reused 24 (delta 4), pack-reused 0 (from 0)
Receiving objects: 100% (28/28), 6.23 KiB | 3.11 MiB/s, done.
Resolving deltas: 100% (6/6), done.
vudinhso@vudinhso-VirtualBox:~/esp32_sketch$ vscode .
vscode: command not found
vudinhso@vudinhso-VirtualBox:~/esp32_sketch$ code .
vudinhso@vudinhso-VirtualBox:~/esp32_sketch$ S
```

Figure 18: `git clone` the online repository and launch `vscode` with `"code ."`

The terminal can be open in VSCode as well. From here, you may clone the code from GitHub:

Git clone <https://github.com/ROS2-AutoBot/01-LEDBlinking.git>

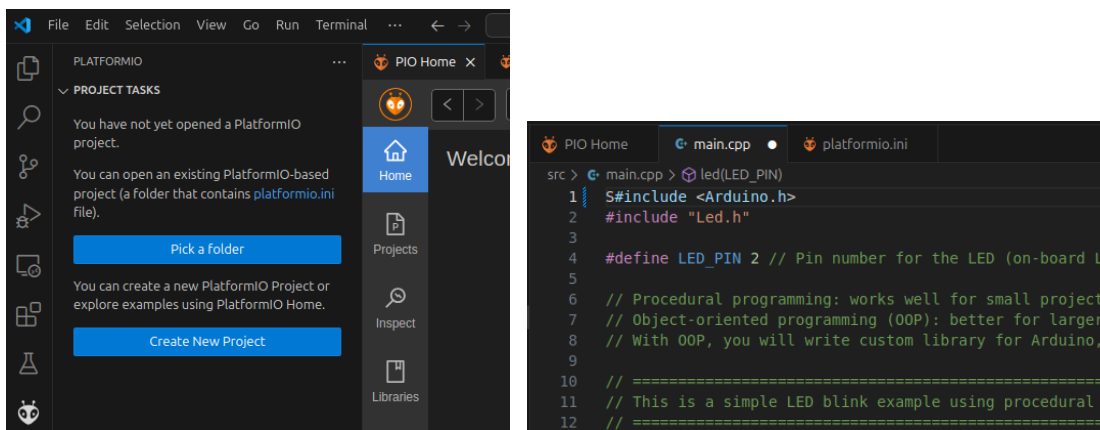


Figure 19: Pick the folder that has just been cloned. You may now build and compile the project

7.2. Arduino sketch for ROS2

The Arduino performs the following tasks:

- Read the serial com for motor cmd velocity
- Performs the PID to run the motor to the correct velocity
- Send the velocity estimated from the encoder value
- Optional: send the IMU data

The information sent on the serial com is up to you. You may send the information on the serial port naively such as:

```
Serial.print(w1)
```

```
Serial.print(w2)
```

...

However, it would be better to:

- Have delimiter in-between data: w1 \t w2 \t w3 \t w4
- Starting and ending delimiter <>: < w1 \t w2 \t w3 \t w4 >
- Have a header: <state: w1 \t w2 \t w3 \t w4>
 <imu: anglex \t angley \t anglez>

The serial communication is highly not reliable, this is why this type of delimiters is useful. CANBUS used in automotive industry would be much better, but it is harder to debug. The Arduino sketch should look like:

```
Serial.print('<');
Serial.print('state:');
Serial.print(w1);
Serial.print("\t");
Serial.print(w2);
Serial.println('>');
```

A code should be written in order to read the serial communication and the speed command from the ros2 hardware interface.

7.3. Writing the hardware interface of ros2_control

Get inspired from existing repo and adapt hardware interface to your need:

git clone <https://github.com/AntoBrandi/Self-Driving-and-ROS-2-Learn-by-Doing-Odometry-Control.git>

It would be possible to write a node that reads and write on the serial communication. However, hardware interface and ros2_control uses the concept of lifecycle node, which works as a state machine. This lifecycle node is useful when the node is initializing, waiting for data, or deconstructed. The effort to write a hardware interface is medium, but can be alleviate, as only the read/write function is important. The rest can be copied/pasted.

Package modification:

- Description:
 - o rmitbot.urdf.xacro


```
<xacro:arg name="is_sim" default="true"/>
```

 This variable is added to select the real robot or the gazebo simulated robot.
 - o rmitbot_ros2_control.xacro
 using simulation or real robot for the hardware interface
- Controller: rmitbot_controller.yaml: use_sim_time: true/false depending on the real/simulated hardware
- Localization: ekf.yaml: use_sim_time: true/false depending on the real/simulated hardware
- Firmware: new package
 - o Arduino_firmware: the sketch to be flashed on the Arduino
 - o Include/rmitbot_firmware/rmitbot_interface.hpp: header file for the cpp code.
Function and variable declaration

7.4. Launching the robot

The robot can be in simulation or in real life.

7.4.1. Launching robot simulation

Use the following command:

```
ros2 launch rmit_bringup simrobot.launch.py
```

This launch:

gazebo
controller
teleopkeyboard

optional: launch robot_localization as well

This bringup pkg allows to launch several launching file.

7.4.2. Launching robot in real life

Launch the real robot

```
ros2 launch rmit_bringup realrobot.launch.py
```

This launch:

hardware_interface
controller
teleopkeyboard

You can subscribe to /joint_states to see the data from the Arduino

```
ros2 topic echo /joint_states
```

optional:

launch robot_localization as well
get the imu
get gazebo working as well, to have sim + real robot working together

topic explanation

/joint_states: wheel velocity and position
/rmitbot_controller/cmd_vel : twist command (Cartesian velocity command)
Motor velocity command are usually not published.

The hardware connection works well, but there is a lot of inertia (?)

8. rmitbot_mapping package: slam

The lidar and slam is introduced in simulation and in hardware.

Install the package for the lidar hardware (rplidar A1M8):

```
sudo apt install ros-jazzy-rplidar-ros
```

Install the slam toolbox:

```
sudo apt install ros-jazzy-slam-toolbox
```

8.1. Lidar and slam in simulation

The modification to add the lidar are as follows:

- In the gazebo.launch.py, in the node ros_gz_bridge: added LaserScan messages
- Added laser_link.STL
- Added rmitbot_laser.xacro
- Having obstacles in the gazebo simulation is essential for the map to be built. If gazebo does not have any world yet, then put one rectangle in the scene.

8.2. lidar and slam in real hardware

Install the package for the lidar:

```
sudo apt install ros-jazzy-rplidar-ros
```

Run the executable and run rviz

```
ros2 run rplidar_ros rplidar_composition --ros-args -p serial_port:=/dev/ttyUSB0 -p
frame_id:=lidar_link -p angle_compensate:=true -p scan_mode:=Standard
ros2 run rviz2 rviz2
```

In fixed frame, write lidar_link. Add laser scan. Select topic /scan. Increase size to 0.03m.

Launch lidar and mapping

```
ros2 launch rmit_localization lidar.launch.py
```

```
ros2 launch ... (slamtoolbox launch file to do)
```

online asynchronous setting: live data, in an asynchronous fashion (we may miss one scan occasionally due to data processing)

Create a config folder and a config file.

Tasks done

- Add a new lidar in the robot_description urdf.xacro (loading stl file graphics)
- Add gazebo.xacro information (add plugin lidar)

9. rmitbot_navigation package: planning, obstacle avoidance

Install nav2 package:

```
sudo apt install ros-jazzy-navigation2 ros-jazzy-nav2-bringup -y
```

If the robot is similar to a turtle bot (i.e., diff drive controller), then launch

```
ros2 launch nav2_bringup navigation_launch.py use_sim_time:=true
```

Add a new map (on top of the map created by mapping), with the topic global costmap.

Select the two goals (on top of rviz) and select where you want to go.

/goal_pose can be published, whether the point is located inside or outside the map

Apparently, /cmd_vel from nav2 stack only generates /cmd_vel when the /goal_pose is within the map created.

The robot AND the goal_pose must be inside the costmap area. (you cannot send the robot in the fog of war).

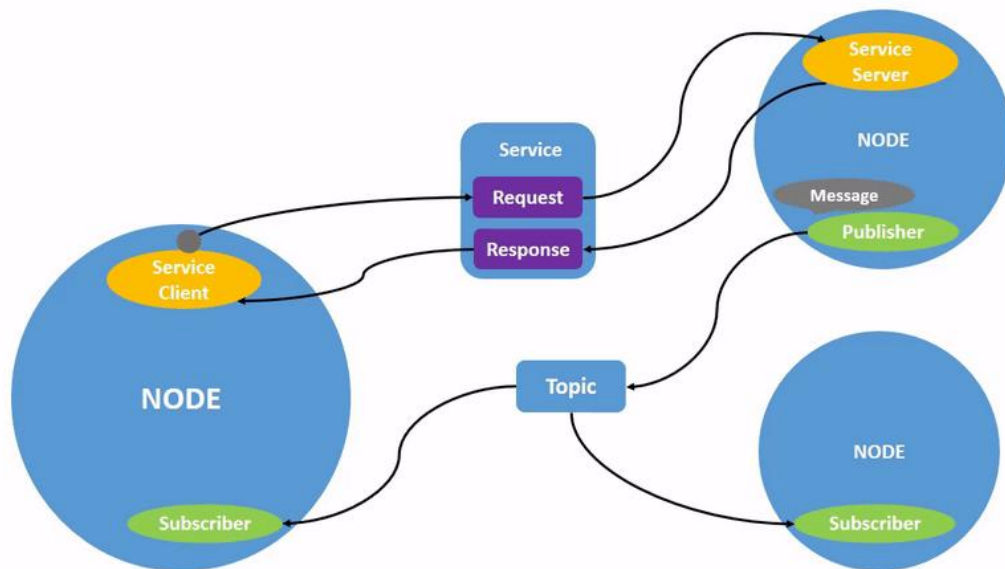
Make sure to modify the yaml file with base_footprint instead of base_link (which is by default in the yaml file)

10. Tutorial Command Line Interface (CLI) – optional

CLI stands for command line interface: we are writing everything in the console (like an animal).

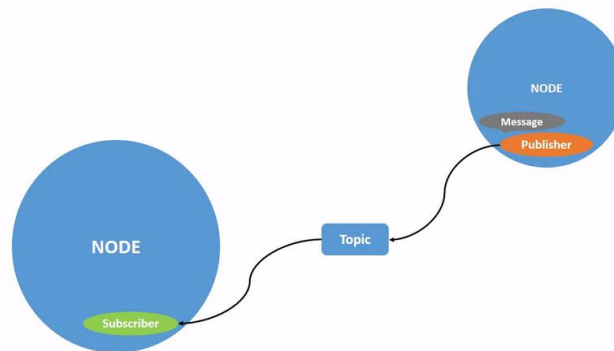
10.1. Nodes

Each node in ROS should be responsible for a single, modular purpose, e.g. controlling the wheel motors or publishing the sensor data from a laser rangefinder. Each node can send and receive data from other nodes via topics, services, actions, or parameters.



<code>ros2 run <package_name> <executable_name></code> <code>ros2 run turtlesim turtlesim_node</code> <code>ros2 run turtlesim turtle_teleop_key</code>	Launch a node (piece of code)
<code>ros2 node list</code>	Show you the names of all running nodes
<code>ros2 run turtlesim turtlesim_node --ros-args --</code> <code>remap __node:=my_turtle</code>	Remapping: reassign default node properties (e.g. name)
<code>ros2 node info <node_name></code> <code>ros2 node info /my_turtle</code>	Access more information

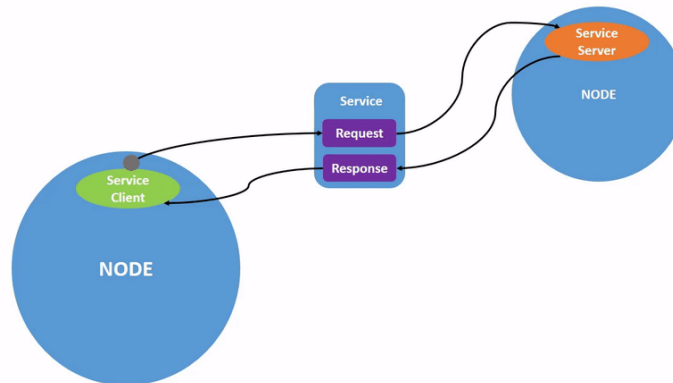
10.2. Topics



<code>rqt_graph</code>	Graphical interface: shows node and topics
<code>ros2 topic list</code> <code>ros2 topic list -t</code>	List of the topic, with their message type
<code>ros2 topic echo <topic_name></code> <code>ros2 topic echo /turtle1/cmd_vel</code>	See data being published in a topic
<code>ros2 topic info <topic_name></code> <code>ros2 topic info /turtle1/cmd_vel</code>	list the number of publisher and subscriber on the topic
<code>ros2 interface show <msg_type></code> <code>ros2 interface show geometry_msgs/msg/Twist</code>	learn details about a message type
<code>ros2 topic pub <topic_name> <msg_type></code> '<args>' <code>ros2 topic pub /turtle1/cmd_vel</code> <code>geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"</code> <code>ros2 topic pub --once -w 2 /turtle1/cmd_vel</code> <code>geometry_msgs/msg/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"</code>	publish manually on the topic from the terminal
<code>ros2 topic hz /turtle1/pose</code> <code>ros2 topic bw /turtle1/pose</code>	Frequency and bandwidth of topic
<code>ros2 topic find <topic_type></code> <code>ros2 topic find geometry_msgs/msg/Twist</code>	List a list of available topics of a given type

10.3. Services

Services are based on a call-and-response model versus the publisher-subscriber model of topics. While topics allow nodes to subscribe to data streams and get continual updates, services only provide data when they are specifically called by a client.



ros2 service list ros2 service list -t	return a list of all the services currently active
ros2 service type <service_name> ros2 service type /clear	To find out the type of a service
ros2 service info <service_name> ros2 service info /clear	To see information of a particular service
ros2 service find <type_name> ros2 service find std_srvs/srv/Empty	to find all the services of a specific type
ros2 interface show <type_name> ros2 interface show std_srvs/srv/Empty ros2 interface show turtlesim/srv/Spawn	to know the structure of the input arguments
ros2 service call <service_name> <service_type> <arguments> ros2 service call /clear std_srvs/srv/Empty ros2 service call /spawn turtlesim/srv/Spawn "{x: 2, y: 2, theta: 0.2, name: \"}"	can call a service
- ros2 service echo <service_name service_type> <arguments> - ros2 launch demo_nodes_cpp introspect_services_launch.py - ros2 param set /introspection_service service_configure_introspection contents - ros2 param set /introspection_client client_configure_introspection contents	To see the data communication between a service client and a service server

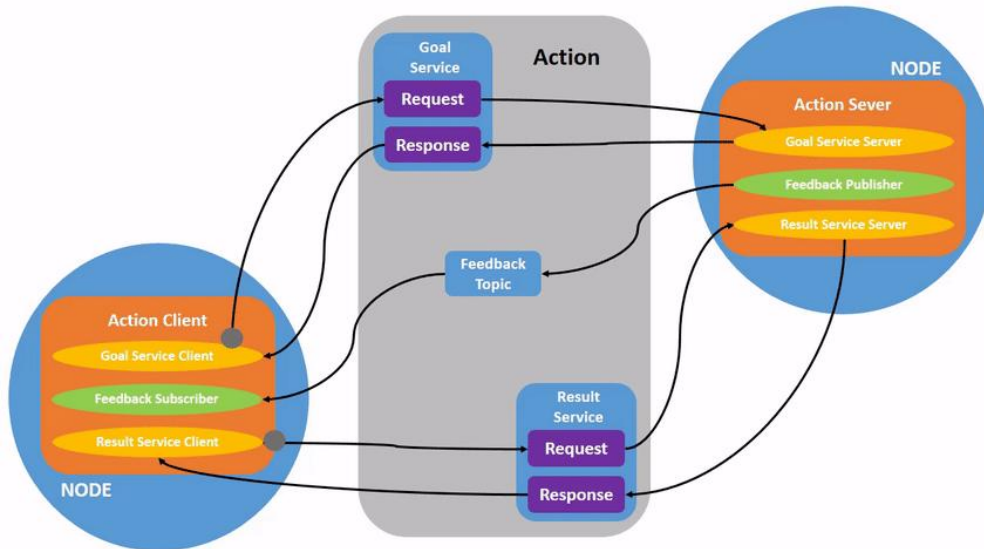
10.4. Parameters

A parameter is a configuration value of a node. You can think of parameters as node settings.

ros2 param list	To see the parameters belonging to your nodes
ros2 param get <node_name> <parameter_name> ros2 param get /turtlesim background_g	To display the type and current value of a parameter
ros2 param set <node_name> <parameter_name> <value> ros2 param set /turtlesim background_r 150	To change a parameter's value at runtime
ros2 param dump <node_name> ros2 param dump /turtlesim > turtlesim.yaml	view all of a node's current parameter
ros2 param load <node_name> <parameter_file> ros2 param load /turtlesim turtlesim.yaml	load parameters from a file
ros2 run <package_name> <executable_name> --ros-args --params-file <file_name> ros2 run turtlesim turtlesim_node --ros-args --params-file turtlesim.yaml	To start the same node using your saved parameter values

10.5. Actions

They consist of three parts: a goal, feedback, and a result. Their functionality is similar to services, except actions can be canceled. They also provide steady feedback, as opposed to services which return a single response.



<code>ros2 node info /turtlesim</code> <code>ros2 node info /teleop_turtle</code>	Node information (sub, pub, service, etc.)
<code>ros2 action list</code> <code>ros2 action list -t</code>	To identify all the actions in the ROS graph
<code>ros2 action type /turtle1/rotate_absolute</code>	to check the action type for the action
<code>ros2 action info /turtle1/rotate_absolute</code>	further introspect
<code>ros2 interface show turtlesim/action/RotateAbsolute</code>	the structure of the action type
<code>ros2 action send_goal <action_name> <action_type> <values></code> <code>ros2 action send_goal /turtle1/rotate_absolute</code> <code>turtlesim/action/RotateAbsolute "{theta: 1.57}"</code> <code>ros2 action send_goal /turtle1/rotate_absolute</code> <code>turtlesim/action/RotateAbsolute "{theta: -1.57}" --feedback</code>	send an action goal with feedback

11. Plotjuggler for data visualization – optional

Information: <https://index.ros.org/p/plotjuggler/>

Tutorial: <https://youtu.be/MnMGjvYxlUk?si=3zabX6mJ8EmFIR0X>

Install ros package: `sudo apt install ros-jazzy-plotjuggler-ros`

Launch plotjuggler: `ros2 run plotjuggler plotjuggler`

Example:

- In a first terminal, launch the teleoperation keyboard node:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

- In a second terminal, launch plotjuggler:

```
ros2 run plotjuggler plotjuggler
```

- In the third terminal, type “ros2 node list”. You should see:

```
/plotjuggler
```

```
/teleop_twist_keyboard
```

- In the third terminal, type “ros2 topic list”. You should see:

```
/cmd_vel → This is the topic from the node /teleop_twist_keyboard
```

```
/parameter_events → This is by default used by ROS2
```

```
/rosout → This is by default used by ROS2
```

- In plotjuggler, chose the correct topic to stream and to visualize the data.

12. Appendix

12.1. Useful terminal script for ros2

Run a package in ros2

```
ros2 run <pkg_name>
```

Run a launchfile in ros2

```
ros2 launch <pkg_name> <launch_file>
```

install packages

```
sudo apt install <pkg_name>
```

Install a package that is part of ros2 core packages:

```
sudo apt install ros-jazzy-<pck_name>
```

Stop ubuntu machine (relevant while using ssh)

```
sudo halt
```

Update your system

```
sudo apt update && sudo apt upgrade -y
```

12.2. Tutorial SSH

Setting up your network for ROS

Preferably use a travel router

Install OpenSSH on both dev machine and RPI.

ip addr: get ip address of your machine

ssh ID@IP access to the rpi remotely via ssh

e.g.: ssh v120506@192.168.47.171

12.3. Tutorial to OOP

https://www.youtube.com/watch?v=cUVryWbVkXk&t=1151s&ab_channel=RoboticsBack-End

```
1#define LED_PIN 13
2
3void setup() {
4  pinMode(LED_PIN, OUTPUT);
5}
6
7void loop() {
8  digitalWrite(LED_PIN, HIGH);
9  delay(500);
10 digitalWrite(LED_PIN, LOW);
11 delay(500);
12}
```

```
42 Led led(LED_PIN);
43
44 void setup() {
45   led.init();
46 }
47
48 void loop() {
49   led.on();
50   delay(500);
51   led.off();
52   delay(500);
53 }
```

Procedural coding

OOP

OOP is better for abstraction and code reuse. However, the code might be longer, but the main code would be much clearer. It is defined as follows:

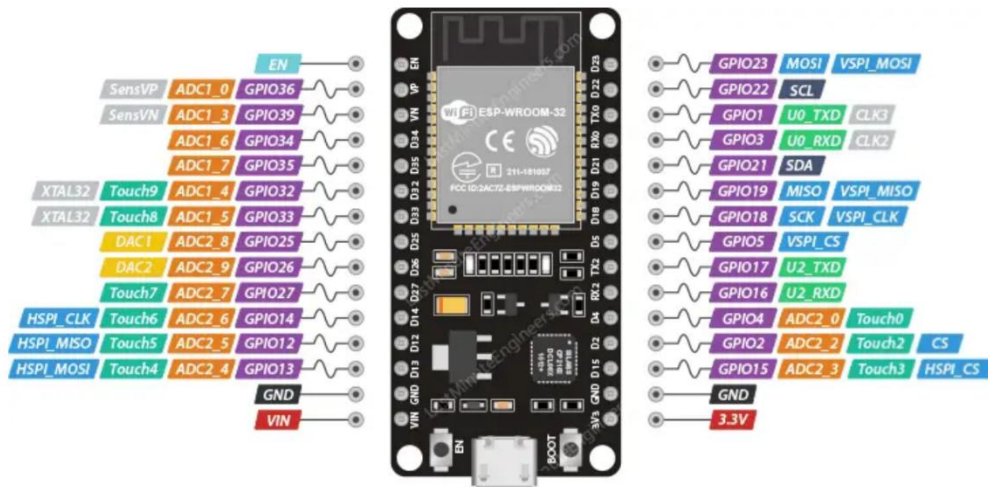
- Class: concatenation of several functions. For example, a motor has the following function:
 - o Init: defining the pin
 - o Run: running the motor with the correct pins
- A class is defined with:
 - o Private Attributes (inside the class only), also known as encapsulation
 - Example: pin used for pwm
 - o Public attributes (inside and outside the class)
 - Constructor (name of the class)
 - this->pin : uses the variable defined in the private attribute
 - method (functionality)
 - void in
 - void off
 - void init
- Object: calling the class. For example, for 4 motors, we will have one class, but four objects.

12.4. Reference and tutorial for using ROS2

1. Getting started with ROS2:
https://www.youtube.com/watch?v=8aoFndU7jos&list=PLubPyrvfL7isH8_VviXuREpXk5J8S_DAT&index=16&ab_channel=MikeLikesRobots
2. Swarm communication: swarm (Open-RMF)
3. Learn ROS 2: Beginner to Advanced Course (Concepts and Code)
https://youtu.be/HJAE5Pk8Nyw?si=X41DTaIPb_VWtRP
4. Very good tutorial:
<https://roboticsbackend.com/how-to-learn-ros2/>
5. AprilTag information and packages:
https://ftc-docs.firstinspires.org/en/latest/apriltag/vision_portal/apriltag_intro/apriltag-intro.html
https://github.com/christianrauch/apriltag_ros
6. Ros mapping project from Northwestern University
<https://github.com/kevin-robb/bebop-eece5550>
- 7.

13. ESP32 pinout

ESP32 S3 Super mini is used instead of ESP32. This is just for reference.



Label	GPIO	Safe to use?	Reason
D0	0	⚠	must be HIGH during boot and LOW for programming
TX0	1	✗	Tx pin, used for flashing and debugging
D2	2	⚠	must be LOW during boot and also connected to the on-board LED
RX0	3	✗	Rx pin, used for flashing and debugging
D4	4	✓	
D5	5	⚠	must be HIGH during boot
D6	6	✗	Connected to Flash memory
D7	7	✗	Connected to Flash memory
D8	8	✗	Connected to Flash memory
D9	9	✗	Connected to Flash memory
D10	10	✗	Connected to Flash memory
D11	11	✗	Connected to Flash memory
D12	12	⚠	must be LOW during boot
D13	13	✓	
D14	14	✓	
D15	15	⚠	must be HIGH during boot, prevents startup log if pulled LOW

D15	15	⚠	must be HIGH during boot, prevents startup log if pulled LOW
RX2	16	✓	
TX2	17	✓	
D18	18	✓	
D19	19	✓	
D21	21	✓	
D22	22	✓	
D23	23	✓	
D25	25	✓	
D26	26	✓	
D27	27	✓	
D32	32	✓	
D33	33	✓	
D34	34	⚠	Input only GPIO, cannot be configured as output
D35	35	⚠	Input only GPIO, cannot be configured as output
VP	36	⚠	Input only GPIO, cannot be configured as output
VN	39	⚠	Input only GPIO, cannot be configured as output



Proposal for pin connection between motor, driver, and ESP32.

Hardware	ESP32
ENC1_A	23
ENC1_B	22
ENC2_A	21
ENC2_B	19
MOT1_A	16
MOT1_B	4
MOT2_A	18
MOT2_B	17

14. ESP32 and MicroROS – optional

RPI4 handles high level computing (navigation and computer vision). ESP32 handles the fast, low-level computing: PID, motor command, sensor data acquisition.

Tutorial available: https://github.com/micro-ROS/micro_ros_arduino/tree/jazzy/examples

Since PlatformIO (PIO) is used, tools related to MicroROS for PIO should be involved instead, but it does not work well nor compile. The link is for reference:

https://github.com/micro-ROS/micro_ros_platformio

14.1. Starting a project in PIO and ESP32

Check the following tutorial (follow the Ubuntu section):

<https://randomnerdtutorials.com/vs-code-platformio-ide-esp32-esp8266-arduino/>

When creating a new project, select **Espressif ESP32 Dev Module**

In PIO, the project architecture is a bit different from Arduino IDE. Please refer to the tutorial to start project with LED blinking, etc.

<https://dronebotworkshop.com/platformio/>

14.2. MicroROS architecture

MicroROS works well with the VSCode, PIO, ESP32, and Arduino framework:

https://github.com/micro-ROS/micro_ros_arduino

MicroROS is a library that must be imported in the esp32 in the platformio.ini. It prepares the communication between the microcontroller and the on-board computer. It allows the esp32 to communicate with the on-board computer via nodes, topics, services, actions.

On the on-board computer, one must run an Agent, which is the communication interface between the MicroROS microcontroller and the ROS2 system. More information in:

<https://youtu.be/Nf7HP9y6Ovo?si=OykCYBySdw16kc55>



Some libraries must be installed in VSCode, please do as follows:

- Open a terminal in VSCode
- Install git, cmake, and python: `sudo apt install -y git cmake python3-pip`
- Install Python interpreter: `sudo apt install python3-venv`

After flashing the code (publisher - subscriber) on the esp32, you may want to see if the communication is enabled.

- How to listen to a node:
 - o Launch the docker agent

`docker run -it --rm -v /dev:/dev -v /dev/shm:/dev/shm --privileged --net=host microros/micro-ros-agent:jazzy serial --dev /dev/ttyUSB0 -v6`

- o Press and release enable button on esp32 (if necessary)
- o List all the topics available: `ros2 topic list -t`
- o Listen to the topic: `ros2 topic echo /micro_ros_arduino_node_publisher`
- o Publish on the topic:


```
ros2 topic pub /int32_subscriber std_msgs/Int32 "{data: 123}"
```

Check your USB device:

```
lsusb
ls /dev/serial/by-id
```

More examples can be found as follows:

https://github.com/micro-ROS/micro_ros_arduino/tree/jazzy/examples

14.3. MicroROS with linorobot

The project linorobot is very similar to what needs to be achieved.

<https://discourse.ros.org/t/linorobot2-diy-ros2-robots-2wd-4wd-mecanum-w-nav2-and-slamtoolbox/22408>

<https://github.com/linorobot/linorobot2>

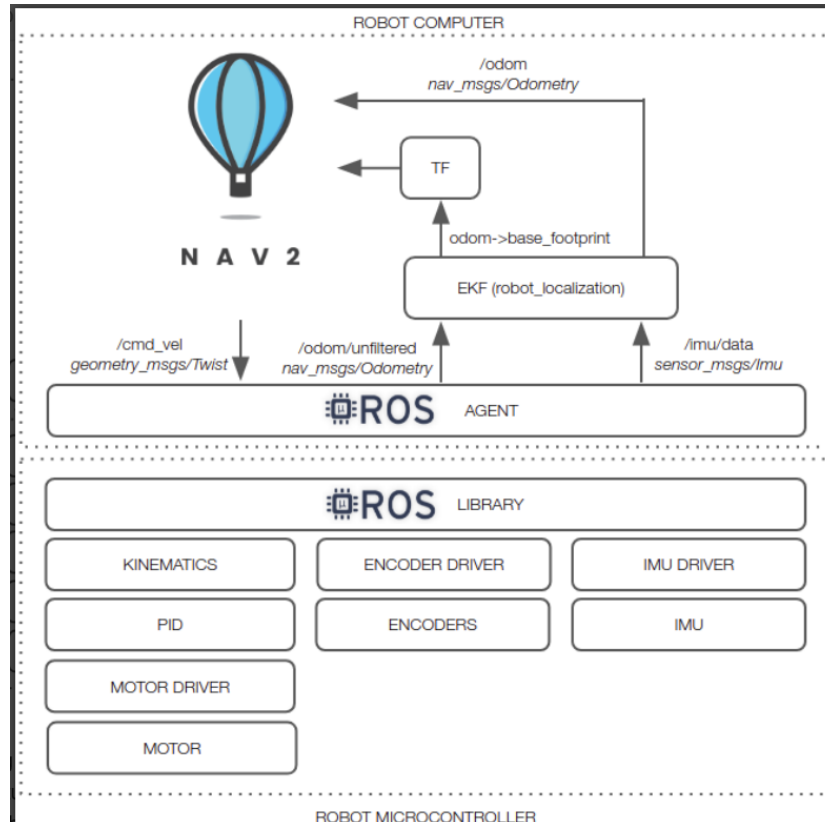


Figure 20: Workflow example: <https://linorobot.org/>

The microcontroller esp32:

- Publishes, i.e., outputs, the following information: IMU and odometry (encoder ticks)
- Subscribes, i.e., receives as an input, a twist message (linear and angular velocities)

How to use keyboard twist (for input command from the keyboard)

- If you need to install it (normally it comes directly with ROS2)
`sudo apt-get install ros2-teleop-twist-keyboard`
- You will probably need to remap it to publish to the correct topic

`ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args --remap cmd_vel:=<topic_name>`

15. Setup rpi5 for ros2

Tutorial: https://youtu.be/Cw_34fuve6E?si=YpOpLHJ70R5byKj1

- select ubuntu server 24.04.2 LTS 64bits
- enable ssh
- set username and password
 - o username v120506-pi
 - o password: same as logging in to v120506
- wifi network: the easiest would be to connect to your personal network, i.e., network from cellphone via hotspot
- you are ready to write on the SDcard
- Go on your phone and find the ipaddr of the rpi4 (normally can be found in the hotspot details)
For me: 192.168.8.171

- In a terminal type:

ssh id@ipaddr

ssh [v120506-pi@192.168.8.171](https://youtu.be/Cw_34fuve6E?si=YpOpLHJ70R5byKj1)

then type the password that you have selected while creating the pi

You are now controlling the rpi4 from your computer via ssh

You may want to update and upgrade your packages

sudo apt update

sudo apt upgrade

sudo reboot → reboot the rpi4 (it closes the ssh connection as well)

exit → end ssh connection

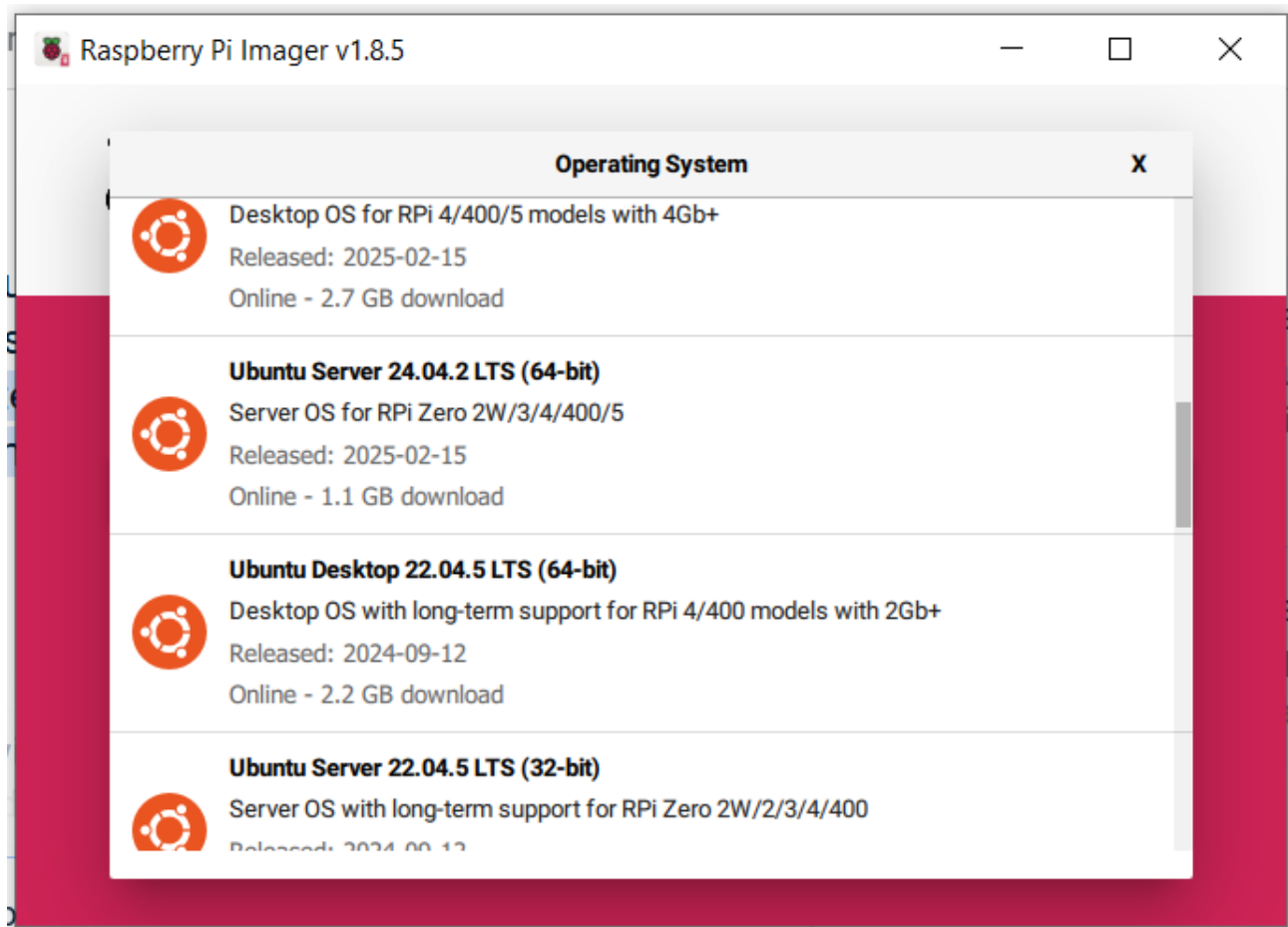
sudo poweroff → turn off rpi4

Then ros2jazzy must be installed through ssh

<https://youtu.be/TPzRNUSVDCE?si=Ar-2p9LyDupGSQK>

be careful: do not install desktop, as rpi4 is not fit to run rviz/gazebo. Install bare bones only.

If you are using a VM machine, make sure that you are using a bridge connection (network option), otherwise you can ping, but DDS multicaster will not work (no publish/subscribe between different computers).



16. Setup a joystick with ros2

A bluetooth joystick is recommended, as it is more flexible. Using a VM, bluetooth can be activated in Ubuntu or Windows, but not both.

Install the joystick package

```
sudo apt install ros-jazzy-joy
```

Install the joystick teleoperation package

```
sudo apt install ros-jazzy-joy-teleop (not working)
```

```
sudo apt install ros-jazzy-joy-teleop-twist-joy
```

Install twist_stamper: joystick publishes a twist without a stamp. A stamp is required while using a real hardware, with mapping and all.

```
sudo apt install ros-jazzy-twist-stamper
```

New: install the teleop-tools

```
sudo apt install ros-jazzy-teleop-tools
```

Try if the teleoperation keyboard works

```
ros2 run key_teleop key_teleop
```

Out of the box, it should publish a topic called /key_vel. You can verify with

```
ros2 topic list
```

```
ros2 topic echo /key_vel
```

Verify the message type of the twist. It should be a twist.

```
ros2 topic info /key_vel
```

Using twist mux will become handy to take over navigation

```
sudo apt install ros-jazzy-twist-mux
```

17. setup docker – tba

So far, you had to install:

- VMBox, Ubuntu 24.04, ros2 Jazzy, vscode, git, terminator, etc
- This is time consuming, and what will happen when Ubuntu upgrade to 26.04 and ros2 to another version?

Using Docker allows to run image: it is like running a virtual machine, but everything is already pre-installed (usually by other engineers in robotics). Let's say I would like to run a previous version of ros2 (humble, because it has a lot of support). I can reinstall Ubuntu 22.04, ros2 humble, and all the associated softwares, or I can use the docker image called ros:humble

docker image pull ros:humble → Get the image from the internet

docker run -it ros:humble → Run the docker image: it has ubuntu 22.04 and ros2 humble

More information at:

Articulated Robotics: <https://youtu.be/XcJzOYe3E6M?si=HZpo33TDFA1Cuja0>

<https://articulatedrobotics.xyz/tutorials/docker/docker-overview>

F1 Tenth at UPenn: https://youtu.be/EU-QaO6xTv4?si=LN_v0i6MaEkdJbco